12. Risks and Technical Challenges

The Process Change Impact Simulator involves several technical tasks, including graph analysis, process simulation, database management, and web application development. Some challenges may arise during implementation and testing. I will manage these risks by developing the application in small stages and testing each major function before moving to the next stage.

Risk	Possible impact	Risk management plan
Graph algorithm complexity	Incorrect dependency analysis may identify the wrong affected activities.	I will use NetworkX and test the graph algorithms with simple workflows before testing more complex processes.
Simulation accuracy	The system may produce incorrect results if process conditions are not handled properly.	I will define clear rules for each sample workflow and compare actual simulation results with manually calculated expected results.
Circular dependencies	Cycles may cause problems when determining the correct process execution order.	I will add cycle detection and display a warning when a process contains circular dependencies.
Database consistency	Incorrect relationships between processes, activities, and changes may lead to missing or invalid records.	I will use SQLite foreign keys, input validation, and database tests to maintain consistent records.
Technology integration	Flask, SQLite, NetworkX, and visualization libraries may not work together as expected.	I will integrate the components gradually and test communication between them during development.
Data availability	Real organizational process data may be confidential or difficult to obtain.	I will create synthetic workflows based on publicly documented examples of purchase approval and employee onboarding.
Visualization difficulties	Large dependency graphs may become difficult to read and understand.	I will test different graph layouts and highlight affected activities to make the results easier to review.
Performance limitations	Impact analysis and simulation may take too long for larger workflows.	I will measure response times using workflows of different sizes and improve the algorithms if needed.
Scope expansion	Adding too many features may delay completion of the main project.	I will focus first on process creation, dependency analysis, simulation, and result comparison. Additional features will be considered only after the core functions work correctly.
Testing and debugging	Errors discovered late in development may delay the final submission.	I will write unit tests during implementation, test the integrated application regularly, and keep time for final corrections.

The main risk is producing correct results when a change affects several connected activities or decision paths. To reduce this risk, I will begin with a simple purchase approval workflow and gradually introduce branching conditions and multiple approvals. I will also keep the original process separate from each proposed change so that results can be compared without losing the original data.

13. Development Schedule

The final project is due on December 8, 2026. I will divide the work into milestones so that the architecture, core features, testing, and documentation can be completed before the submission date. The schedule below is a proposed plan for the remaining development work.

Milestone	Planned dates	Main activities and expected outcome
1. Architecture and Design	October 28–30, 2026	Finalize the system architecture, database design, functional requirements, and technology choices.
2. Technical Proof of Concept	October 31–November 2, 2026	Demonstrate graph construction, downstream dependency analysis, basic simulation, and comparison using a sample workflow.
3. Core Implementation	November 3–10, 2026	Implement process management, activity creation, dependency management, change scenarios, and database operations.
4. Component Integration	November 11–14, 2026	Connect Flask routes, SQLite, NetworkX analysis, simulation logic, and result visualization.
5. Minimum Viable Product (MVP)	November 15–18, 2026	Prepare a working application that supports process setup, change selection, impact analysis, simulation, and result review.
6. Testing and Evaluation	November 19–23, 2026	Run unit, integration, API, performance, and functional tests using synthetic workflows. Record test results and identify problems.
7. Refinement	November 24–27, 2026	Correct defects, improve graph readability, handle invalid inputs, and refine simulation results.
8. Documentation	November 28–December 2, 2026	Complete technical documentation, testing evidence, screenshots, setup instructions, and the final project report.
9. Final System and Presentation	December 3–8, 2026	Perform final checks, verify installation instructions, prepare the demonstration, organize source files, and submit the completed project.

Schedule management

I will review progress at the end of each milestone and check whether the planned functions are working. If a task takes longer than expected, I will prioritize the main features before adding optional improvements. The core application must be able to represent a process, identify downstream effects, simulate a proposed change, and compare the results.

I will reserve the final days for checking the application, preparing screenshots, and verifying that the source code and documentation are ready for submission. This schedule should help

me complete the project by December 8, 2026, while leaving time to correct important issues found during testing.